

CS240 Algorithm Design and Analysis

Spring 2026

Problem Set 2

Due: 23:59, April 12, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

Given a sequence of integers $a_1, a_2, \dots, a_N$, an increasing subsequence is a subsequence $a_{i_1}, a_{i_2}, \dots, a_{i_k}$, such that $1 \leq i_1 < i_2 < \dots < i_k \leq N$ and $a_{i_1} < a_{i_2} < \dots < a_{i_k}$. Your task is to find the maximum possible sum among all increasing subsequences of the given sequence.

Note that the longest increasing subsequence does not necessarily have the largest sum. Provide the state transition equation and the corresponding pseudocode.

Example 1:

Input: $a = [1, 7, 3, 5, 9, 4, 8]$

Output: 18

Explanation: One optimal increasing subsequence is $(1, 3, 5, 9)$, whose sum is 18.

Example 2:

Input: $a = [100, 1, 2, 3]$

Output: 100

Explanation: Although $(1, 2, 3)$ is longer, its sum is only 6. The subsequence (100) has the maximum sum.

Solution:

We can solve this problem using dynamic programming. Let $dp[i]$ represent the maximum sum of an increasing subsequence that ends at index i .

When calculating $dp[i]$, we consider the following cases:

1.Base Case: The subsequence contains only $a[i]$. Therefore,

$$dp[i] = a[i].$$

2.Transition Case: For every $j < i$, if $a[j] < a[i]$, then $a[i]$ can be appended to an increasing subsequence ending at j . In this case, we update:

$$dp[i] = \max(dp[i], dp[j] + a[i]).$$

Thus, the complete state transition equation is:

$$dp[i] = \max \left(a[i], \max_{0 \leq j < i, a[j] < a[i]} (dp[j] + a[i]) \right).$$

The final answer is:

$$\max_{0 \leq i < N} dp[i].$$

The state $dp[i]$ stores the maximum sum of any increasing subsequence ending at position i .

Any such subsequence must be one of the following:

It consists only of $a[i]$, giving sum $a[i]$ or it is formed by appending $a[i]$ to an increasing subsequence ending at some earlier position j , where $a[j] < a[i]$.

Since we examine all valid previous positions j , and take the maximum among all possibilities, $dp[i]$ correctly computes the best sum for an increasing subsequence ending at i .

Finally, the optimal increasing subsequence may end at any position, so taking the maximum over all $dp[i]$ gives the correct answer.

Algorithm 1 Maximum Sum Increasing Subsequence

Input: Array $a[0 \dots n - 1]$

Output: Maximum sum of an increasing subsequence

```
1:  $dp \leftarrow$  array of size  $n$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $dp[i] \leftarrow a[i]$ 
4:   for  $j \leftarrow 0$  to  $i - 1$  do
5:     if  $a[j] < a[i]$  then
6:        $dp[i] \leftarrow \max(dp[i], dp[j] + a[i])$ 
7:     end if
8:   end for
9: end for
10:  $ans \leftarrow dp[0]$ 
11: for  $i \leftarrow 1$  to  $n - 1$  do
12:    $ans \leftarrow \max(ans, dp[i])$ 
13: end for
14: return  $ans$ 
```

Problem 2:

A student plans to select courses for the coming semester. There are m course groups, and each group contains several candidate courses. We denote a course by $X(c, v)$, where X is the course name, c denotes its credit cost, and v denotes its benefit value.

The student may choose at most one course from each group, and the total number of credits used cannot exceed W . Your task is to determine the maximum total benefit that can be obtained under the credit limit. Provide the dynamic programming formulation and the corresponding pseudocode.

Example 1:

Input: $m = 3$, $W = 5$

Group 1: $A(1, 2), B(3, 4)$

Group 2: $C(2, 2), D(4, 5)$

Group 3: $E(1, 1), F(2, 3), G(3, 4)$

Output: 7

Explanation: An optimal selection is to choose course $A(1, 2)$ from Group 1, course $C(2, 2)$ from Group 2, and course $F(2, 3)$ from Group 3. The total credit cost is $1 + 2 + 2 = 5$, and the total benefit is $2 + 2 + 3 = 7$.

Example 2:

Input: $m = 2$, $W = 4$

Group 1: $A(1, 2), B(2, 3)$

Group 2: $C(2, 4), D(3, 5)$

Output: 7

Explanation: An optimal selection is to choose course $B(2, 3)$ from Group 1 and course $C(2, 4)$ from Group 2. The total credit cost is $2 + 2 = 4$, and the total benefit is $3 + 4 = 7$.

Solution:

We solve this problem using dynamic programming. Since the student may choose at most one course from each group, this is a standard group knapsack problem.

Let $dp[i][w]$ represent the maximum total benefit that can be obtained by considering the first i groups with total credit limit w .

When calculating $dp[i][w]$, we consider the following cases:

1.Base Case: If no group is considered, then no course can be selected. Therefore,

$$dp[0][w] = 0 \quad \text{for all } 0 \leq w \leq W.$$

2. Transition Case: For Group i , we may either choose no course from this group or choose exactly one course from it.

If we choose no course from Group i , then

$$dp[i][w] = dp[i-1][w].$$

If we choose a course with credit cost c and benefit value v from Group i , and if $w \geq c$, then we may use the remaining credit capacity $w - c$ for the first $i - 1$ groups. In this case, we update

$$dp[i][w] = \max(dp[i][w], dp[i-1][w-c] + v).$$

Thus, the complete state transition equation is:

$$dp[i][w] = \max \left(dp[i-1][w], \max_{\text{course } (c,v) \text{ in Group } i, c \leq w} (dp[i-1][w-c] + v) \right).$$

The final answer is:

$$dp[m][W].$$

The state $dp[i][w]$ stores the maximum total benefit that can be obtained from the first i groups under credit limit w .

Any feasible solution for the first i groups must be one of the following:

It selects no course from Group i , in which case the total benefit is $dp[i-1][w]$, or it selects exactly one course from Group i , say a course with credit cost c and benefit value v , in which case the total benefit is $dp[i-1][w-c] + v$.

Since we examine all valid choices from Group i and take the maximum among all possibilities, $dp[i][w]$ correctly computes the best total benefit for the first i groups under credit limit w .

Finally, after all m groups have been processed, $dp[m][W]$ gives the correct answer.

Algorithm 2 Course Selection with Group Constraints

Input: m course groups and total credit limit W

Output: Maximum total benefit under the credit limit

```
1: create array  $dp[0 \dots m][0 \dots W]$ , initialized with 0
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $w \leftarrow 0$  to  $W$  do
4:      $dp[i][w] \leftarrow dp[i-1][w]$ 
5:     for each course  $(c, v)$  in Group  $i$  do
6:       if  $w \geq c$  then
7:          $dp[i][w] \leftarrow \max(dp[i][w], dp[i-1][w-c] + v)$ 
8:       end if
9:     end for
10:  end for
11: end for
12: return  $dp[m][W]$ 
```

Problem 3:

On a wall composed of a line of grid cells, there is a painting robot. The robot follows these two painting rules:

1. The robot can only paint using one color at a time, and it can only paint a continuous area of the same color.
2. The robot can start painting from any grid cell and can overwrite areas that have been previously painted.

Given a string s , where each character represents the color of a grid cell (e.g., “r” for red, “b” for blue, “g” for green), your task is to calculate the minimum number of painting operations needed for the robot to cover all the grid cells. Provide the state transition equation and the corresponding pseudocode.

Example 1:

Input: $s = \text{“rrrbbb”}$ Output: 2

Explanation: First, paint the red area “rrr”, then paint the blue area “bbb”.

Example 2:

Input: $s = \text{“rbr”}$ Output: 2

Explanation: First, paint the entire row with red “rrr”, then overwrite the middle cell with blue “b”.

Solution:

We can solve this problem using dynamic programming. Let $f[i][j]$ represent the minimum number of painting operations required to complete the section from index i to j .

When calculating $f[i][j]$, we consider the following cases:

1. Boundary Condition: If $i = j$, meaning the section has only one grid cell, then only one painting operation is required:

$$f[i][i] = 1.$$

2. Same Color: If $s[i] = s[j]$, meaning the colors at both ends are the same, the robot can cover $s[j]$ while painting $s[i]$. Therefore, we only need to consider how to complete the section from i to $j - 1$, giving:

$$f[i][j] = f[i][j - 1].$$

We do not need to worry about the exact painting plan for section $[i, j - 1]$ because we can always complete $s[i]$ first and cover $s[j]$ simultaneously.

3. Different Colors: If $s[i] \neq s[j]$, meaning the colors at both ends are different, the robot needs to separately complete the left and right segments.

We denote these segments as $[i, k]$ and $[k + 1, j]$, where $i \leq k < j$. In this case:

$$f[i][j] = \min_{k=i}^{j-1} (f[i][k] + f[k + 1][j]) .$$

The complete state transition equations can be summarized as:

$$f[i][j] = \begin{cases} 1, & \text{if } i = j \\ f[i][j - 1], & \text{if } s[i] = s[j] \\ \min_{k=i}^{j-1} (f[i][k] + f[k + 1][j]), & \text{if } s[i] \neq s[j] \end{cases}$$

The final answer will be $f[0][n - 1]$, where n is the length of the string s .

Algorithm 3 Robert Painter

Input: String s of length n

Output: Minimum number of painting operations

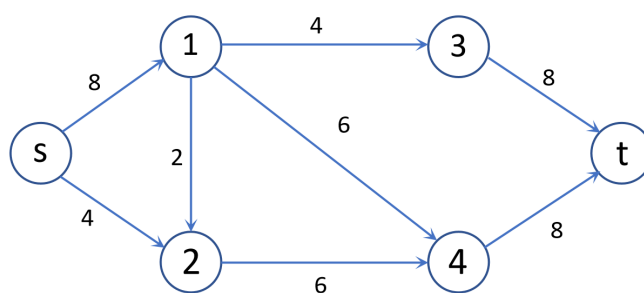
```

1:  $n \leftarrow \text{length of } s$ 
2:  $f \leftarrow$  2D array of size  $n \times n$ , initialized with 0
3: for  $i \leftarrow n - 1$  downto 0 do
4:    $f[i][i] \leftarrow 1$ 
5:   for  $j \leftarrow i + 1$  to  $n - 1$  do
6:     if  $s[i] = s[j]$  then
7:        $f[i][j] \leftarrow f[i][j - 1]$ 
8:     else
9:        $min\_value \leftarrow \infty$ 
10:      for  $k \leftarrow i$  to  $j - 1$  do
11:         $min\_value \leftarrow \min(min\_value, f[i][k] + f[k + 1][j])$ 
12:      end for
13:       $f[i][j] \leftarrow min\_value$ 
14:    end if
15:  end for
16: end for
17: return  $f[0][n - 1]$ 

```

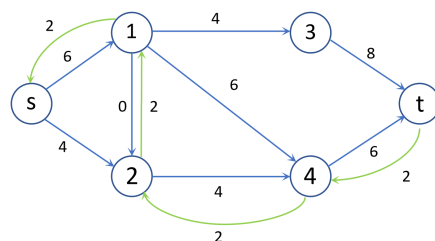
Problem 4:

Execute the Ford-Fulkerson algorithm on the flow network shown in the figure below. Display the residual network after each flow update and determine the maximum flow at the end. During each iteration, choose the lexicographically smallest available augmenting path. For example, if two augmenting paths are $s \rightarrow 2 \rightarrow t$ and $s \rightarrow 3 \rightarrow t$, select $s \rightarrow 2 \rightarrow t$ since it is lexicographically smaller. Likewise, if the paths $s \rightarrow 3 \rightarrow t$ and $s \rightarrow 2 \rightarrow 4 \rightarrow t$ are available, choose $s \rightarrow 2 \rightarrow 4 \rightarrow t$.

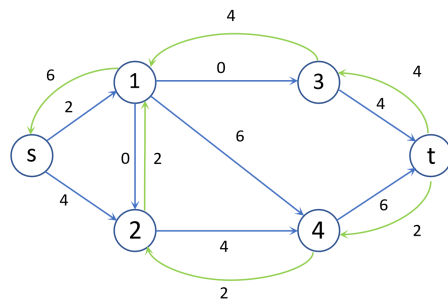


Solution:

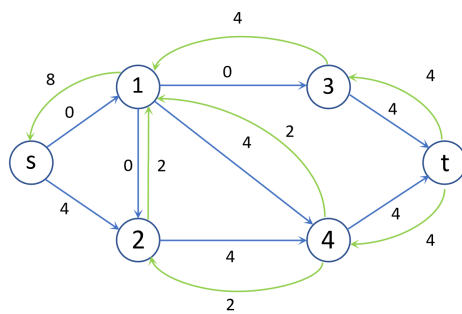
Step 1: the augmenting path is $s \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow t$. The capacity of this path is $\min(8, 2, 6, 8) = 2$.



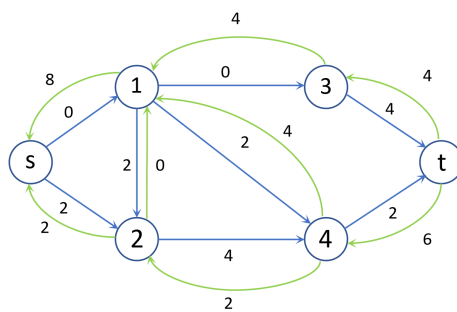
Step 2: the augmenting path is $s \rightarrow 1 \rightarrow 3 \rightarrow t$. The capacity of this path is $\min(6, 4, 8) = 4$.



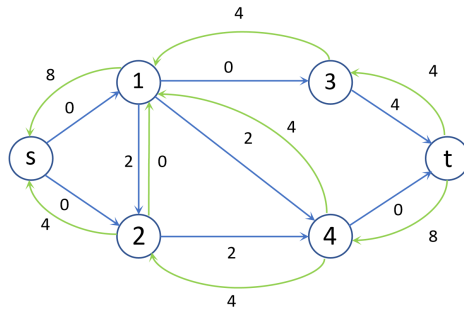
Step 3: the augmenting path is $s \rightarrow 1 \rightarrow 4 \rightarrow t$. The capacity of this path is $\min(2, 6, 6) = 2$.



Step 4: the augmenting path is $s \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow t$. The capacity of this path is $\min(4, 2, 4, 4) = 2$.



Step 5: the augmenting path is $s \rightarrow 2 \rightarrow 4 \rightarrow t$. The capacity of this path is $\min(2, 4, 2) = 2$.



The final maximum flow = 12.

Problem 5:

Before the beginning of a new semester at ShanghaiTech University, there are n students who need to register for courses and m available courses. The registration process is subject to the following constraints:

1. Eligibility constraint: Each student may only register for courses that are allowed under their major requirements.
2. Student course limit: Each student i may enroll in at most k_i courses during the semester.
3. Course capacity limit: Each course j has a capacity limit and can admit at most c_j students.

Your task is to formulate this problem as a network flow model and explain the maximum number of valid student-course assignments subject to the constraints above.

Solution:

Graph Construction

We first define the vertices of our directed graph $G = (V, E)$. We introduce a source node S and a sink node T . We also create a set of student nodes representing the n students, and a set of course nodes representing the m courses.

Next, we define the directed edges and their capacities to satisfy the constraints.

Student Capacity Constraint: To satisfy the student course limit, we add a directed edge from the source S to each student node i . The capacity of this edge is set to k_i . This guarantees that the total flow passing through student i cannot exceed the maximum number of courses they are allowed to take.

Eligibility Constraint: To satisfy the eligibility requirement, we evaluate the major requirements. If student i is eligible to take course j , we add a directed edge from student node i to course node j . The capacity of this edge is set to 1. This binary capacity ensures that a student can enroll in a particular course at most once.

Course Capacity Constraint: To satisfy the course capacity limit, we add a directed edge from each course node j to the sink T . The capacity of this edge is set to c_j . This strict limit ensures that the total flow out of course j does not exceed its maximum allowable enrollment.

With the network fully constructed, the problem simplifies to finding the maximum flow from the source S to the sink T . The value of this maximum flow directly equals the maximum number of valid student-course assignments.

To extract the actual registration results, we simply inspect the edges between the student nodes and the course nodes. An edge from student i to course j carrying a flow of 1 indicates a successful course registration. Algorithms like Dinic's Algorithm or the Edmonds-Karp Algorithm can be used to compute this efficiently.

Problem 6:

Yansen, a student at ShanghaiTech, recently started playing Animal Crossing. The game follows real-time (a week has 7 days).

There are n events, each requiring different materials, and each event is available on a few days of the week. Yansen can complete up to e events per day due to academic and research commitments.

Yansen wants to build a special Gundam and has calculated the materials needed. Starting on Monday, what is the minimum number of days (including rest days) to collect all the materials? (Hint: Network Flow)

Input Description

The first line contains two integers, n, e ($1 \leq n \leq 1000, 1 \leq e \leq 100$), separated by a space, representing the number of different events and the maximum number of events Yansen can complete in a day.

The next n lines, each containing $m_i + 2$ integers $c_i, m_i, a_{i1}, a_{i2}, \dots, a_{im_i}$ ($1 \leq c_i \leq 10^5, 1 \leq m_i \leq 7, 1 \leq a_{ij} \leq 7$, for any $j \neq k, a_{ij} \neq a_{ik}$), represent the required number of completions c_i , and the days of the week when the event is available.

Output Description

Output a single integer representing the minimum number of days Yansen needs to gather all the required materials.

Input:

```
5 2
2 4 1 3 5 7
2 3 2 4 6
2 2 1 2
2 2 3 4
2 2 5 6
```

Output: 5

Explanation:

The solution can proceed as follows:

Day 1 (Monday): Complete events 1 and 3 once each.

Day 2 (Tuesday): Complete events 2 and 3 once each.

Day 3 (Wednesday): Complete events 1 and 4 once each.

Day 4 (Thursday): Complete events 2 and 4 once each.

Day 5 (Friday): Complete event 5 twice.

Exactly 5 days are needed to finish all requirements.

Solution:

Using Binary Search:

We treat the answer (minimum days) as a range and initialize it from $[0, 10^9/e]$, meaning that the minimum is possibly 0 days, and the maximum is $10^9/e$ days.

We use binary search to repeatedly try different values of days, mid , to check whether all event requirements can be met within mid days.

Using Network Flow for Feasibility Checking

For each mid obtained from the binary search, use a network flow algorithm to determine if all events can be satisfied within mid days.

Construct a flow network graph that consists of the source, sink, day nodes, and event nodes, and solve the maximum flow problem.

1. Node Setup

Source node S and sink node T .

Day nodes: representing each day from 1 to 7. These are labeled from 3 to 9.

Event nodes: representing the n events, labeled from $9 + 1$ to $9 + n$.

2. Edge Setup

Connect the source node S to each day node. The capacity represents the maximum number of events that can be completed on that day. The capacity is calculated as:

$$\left(\left\lfloor \frac{mid}{7} \right\rfloor + [mid \bmod 7 \geq i] \right) \times e$$

This means there are $\lfloor \frac{mid}{7} \rfloor$ complete cycles of a week, and if $mid \bmod 7 \geq i$, there will be one extra opportunity on day i .

Connect day nodes to event nodes, representing the days on which each event can be completed. The capacity is set to infinity, indicating that an event can be completed multiple times on a valid day.

Connect each event node to the sink node T . The capacity for these edges is set to the number of times the event needs to be completed, c_i .

3. Checking if the Conditions are Met

Calculate the maximum flow from the source S to the sink T .

If the maximum flow is greater than or equal to the total requirement $\sum c_i$, then it is possible to satisfy all the event requirements within mid days.

If the conditions are met, try reducing the number of days; otherwise, increase the number of days.